

Procedural generation of dungeons' maps and locked-door missions through an evolutionary algorithm validated with players

Leonardo Tortoro Pereira^{*}, Paulo Victor de Souza Prado, Rafael Miranda Lopes, Claudio Fabiano Motta Toledo

Universidade de São Paulo, Instituto de Ciências Matemáticas e de Computação, São Carlos, SP, Brazil

ARTICLE INFO

Keywords:

Evolutionary algorithm
Procedural content generation
Level generation
Mission generation
Adaptive game content

ABSTRACT

The present research introduces an evolutionary algorithm able to procedurally generate dungeon maps containing locked door missions. The evolutionary algorithm evolves a tree structure, which encodes dungeons, aiming to generate levels close to the input configuration provided by a game designer. The tree structure holds information about the number of rooms, connections between them, and their position within a 2D map. The proposed encoding also allows evolving semantic information about the narrative of the game. This is done by feasibly setting keys and locks throughout the dungeons for locked door missions. The generated dungeons are then evaluated computationally and as a proof of concept using an adventure game prototype. A total of 70 players evaluated the contents and the results show that the procedurally generated levels are perceived as more human-made, fun, and difficult than their human-made counterparts for most cases.

1. Introduction

Almost 3 billion people play video games around the globe, a market with an estimated revenue of U\$160 billion by the end of 2020 based on [Newzoo \(2020\)](#). Arguably, missions and play space are the fundamental principles that build these games. Missions are a set of goals that must be accomplished. Play space is the physical layout where the game takes place as reported in [Dormans and Bakkes \(2011\)](#). Action-adventure and role-playing games usually have a dungeon as their space: a set of interconnected rooms containing different challenges ([Dahlskog, Toglioli, & Björk, 2015](#); [Baldwin, Dahlskog, Font, & Holmberg, 2017](#)). These dungeons can provide missions for the players with their layout alone, as the locked door puzzles, common in *The Legend of Zelda* series of games ([Miyamoto & Tezuka, 1986](#))¹. In these games, the players must search for and use tools (or items) that open locked paths to complete their adventure. These are known as spatial awareness puzzles² or spatially situated quests as described in [Calvin and Michael \(2007\)](#) and [Adams \(2009\)](#). Usually, each dungeon and the placement of these puzzles inside them must be done by a human designer.

However, Procedural Content Generation (PCG) can be applied to

create more dungeons in less time, reducing costs and increasing the re-playability value of the game by generating a wide set of dungeons ([Iosup, 2011](#); [Hendriks, Meijer, Van Der Velden, & Iosup, 2013](#); [Games, 2016b](#); [Games, 2016a](#)). Moreover, the generation of multiple contents concurrently is advancing in the PCG field, with researchers aiming to join together different algorithms employing a *maestro* algorithm, able to guarantee the feasibility of the resulting content. Some examples include the generation of levels and sounds, visuals and rules, narrative and levels, and others as reviewed in [Liapis, Yannakakis, Nelson, Preuss, and Bidarra \(2019\)](#).

[Liapis \(2017\)](#) presented an Evolutionary Algorithm (EA) able to generate feasible levels and missions, but only by enforcing many constraints which could become hard to satisfy during the evolution process. These constraints are akin to test conditions that the generated content must pass to be feasible. For example, they test if all tiles are connected, how many tiles of a specific type can be added, how many reward and challenge segments exist, the maximum number of monsters, treasures and exists, if monsters block paths to treasures and others.

We propose a PCG technique based on EA that evolves a population

^{*} Corresponding author.

E-mail addresses: leonardo@usp.br (L.T. Pereira), claudio@icmc.usp.br (C.F.M. Toledo).

¹ The term locked doors is given to obstacles preventing the players' advance through the game until they find a solution. A set of locked doors with switches or keys that open them or grasses that can be cut by blades are examples of such missions. ([Adams, 2009](#), p. 268)

² Spatial awareness puzzles challenge the player to memorize the placement of objects, such as blocked paths and items

of randomly generated dungeon maps filled with locked-door missions, aiming to match the desired level's and mission's characteristics set as input by the game designer. However, our proposal introduces an encoding and decoding process for the EA that facilitate the search for solutions, as well as by evolving them together in the same representation, avoiding the use of a *maestro* algorithm to ensure the feasibility, simplifying the whole process.

The encoding is a tree-structure where we only need to check constraints when creating the level (decoding), and on the specific branches where crossover and mutation occurs, reducing the operations over these procedures. The constraints are if the key-lock puzzle is feasible after the crossover operation and if the rooms can fit in a grid. Other than that, the representation itself guarantees the feasibility. The only other constraint is the number of visited rooms and opened locks, performed once per level by the path-finding algorithms introduced here.

The present work extends the algorithm and preliminary results reported in [Pereira, Prado, and Toledo \(2018\)](#) where a first version of the EA was proposed.

The crossover operator in [Pereira et al. \(2018\)](#) was unable to keep the locks and keys in the encoding, so they were discarded after every exchange between individuals with new locks and keys being placed at random. In the current EA, we present novel crossover and mutation operations that can evolve the semantic data from levels and missions more effectively, without discarding information from the encoding of parent solutions.

The fitness function is also enhanced: the previous EA tried to minimize the difference between the number of rooms, locks, and keys, and the linearity of the level that the user wanted and the ones from the generated level. Now, we add an adaptation of A* and Depth-First Search (DFS) algorithms to simulate the behavior of two different types of players when traversing the level, enabling us to create more challenging puzzles and levels, as we now guarantee a certain level of exploration, unlike in [Pereira et al. \(2018\)](#).

Finally, a game prototype was developed as a proof-of-concept to show that our PCG algorithm may be useful for applications in the game industry. The prototype is applied to test our levels, based on users' evaluations, against those created by professional designers.

We collected data after volunteers played the levels in a game very similar to classic action-adventure ones, allowing a much more precise evaluation of their opinions.

To sum up our contribution we state the following research question: is it possible to generate contents for mission and levels without complex constraints and losing quality during the process? Most related works in Section 2 reported complex constraints when generating levels and missions, and few of them report user case studies from a game prototype to evaluate the quality of such contents. Thus, we answer such research question by introducing an EA able to generate both contents simultaneously, without complex constraints to be satisfied during the EA steps such as encoding (solution representation/genotype) and decoding (phenotype transcription) process, application of evolutionary operators (crossover, mutation) and fitness evaluation. About the possible loss of quality in the process, we conducted an evaluation of the feedback provided by the players from our game prototype using a Likert-scale questionnaire, as reported in works such as [Lora, Sánchez-Ruiz, González-Calero, and Gómez-Martín \(2016\)](#), Turing-like test, as done in [Ferreira and Toledo \(2018\)](#), and statistical analysis with t-test comparing, in pairs, levels generated by our algorithm and equivalent human-authored ones, as recommended by [Winter and Dodou \(2010\)](#). These methodologies allow us to present a more precise idea about subjective quality aspects like difficulty and fun from the generated contents.

Section 2 reviews some related works, while the model of the dungeon used in this research and its creation process are summarized in Section 3. The improved version of the EA, named Constrained EA, is presented in Section 4. The Section 5 introduces our game prototype, and the results achieved from our experiments are reported in Sections 6

Table 1

Comparison of this work's contributions to some of the closest related researches in the recent literature.

Work	LG	MG	#PCG	CC	CT	GP	PT
Adams et al. (2017)	✓	×	1	×	×	✓	7
Khalifa et al. (2016)	✓	×	2	✓	✓	✓	25
Alvarez et al. (2019)	✓	✓	1	✓	✓	✓	×
Font et al. (2016)	✓	✓	2	✓	✓	✓	×
Liapis (2017)	✓	✓	2	✓	✓	✓	×
Green et al. (2019)	✓	✓	2	✓	✓	✓	×
This	✓	✓	1	×	✓	✓	70

and 7. We conclude our work and present possible extensions in Section 8.

2. Related work

A summary of the closest related works is shown in [Table 1](#), based on features explored by our research to advance in level and mission generation using PCG algorithms. Thus, we compare if the literature work uses Level Generation (LG) (✓) or not (×), Mission Generation (MG) or not, how many different algorithms are used to generate the contents (#PCG), if there are Complex Constraints (CC), following [Liapis \(2017\)](#) concept for complex ones or when the authors claim such complexity by themselves. It is also evaluated if any sort of Computational Tests (CT) were conducted, be it to measure the accuracy of the algorithms or their expressive range, as well as if the studies developed any sort of Game Prototype (GP) to validate their solutions. Finally, we analyze if there are Player Tests (PT) to evaluate the contents and, if they were, how many players take part on them. Besides the works summarized in [Table 1](#), this section will also discuss other important landmarks for level and mission in PCG.

The authors in [Adams, Parekh, and Louis \(2017\)](#) apply an iterative EA to evolve cellular automata (CA) rules for procedural level generation of mazes. The iterative EA allows users to evaluate a subset of mazes from the population, where each individual encodes CA rules and the starting map state. A total of seven users evaluates the mazes for ten generations, and they play the best content generated in the initial population, fifth, generation and last generation. The General Video Game Level Generation (GVG-LG) framework is introduced in [Khalifa, Perez-Liebana, Lucas, and Togelius \(2016\)](#). The authors report a pilot study creating levels for Frog, Pac Man, and Zelda games. The Video Game Description Language (VGDL) is employed to code different types of games. GVG-LG has three level generators named random, constructive, and search-based. The search-based generator applies Feasible Infeasible two Population (FI2Pop) Genetic Algorithm, which evolves two different populations with different encoding and fitness function, so we consider as two different PCG algorithms.. The feasible population is evaluated by game simulations, while the infeasible one is evaluated by some constraints that have to be satisfied on the number of sprites and length. The authors in [Adams et al. \(2017\)](#) and [Khalifa et al. \(2016\)](#) apply EA for level generation only, while we advance by generating simultaneously mission and level generation using an EA. Also, both works gathered data from 7 and 25 users, respectively, while our study collected feedback from 70 users that played our game prototype.

The work in [Alvarez, Dahlskog, Font, and Togelius \(2019\)](#) applies PCG with Quality Diversity (PCG-QD) for dungeons generations, where the game MiniDungeons 2 is used to test a mixed-initiative QD algorithm called Constrained MAP-Elites algorithm. The algorithm also applies FI2Pop genetic algorithm with feasible and infeasible populations based on the fulfillment of constraints. The user can edit the level and its rooms during the evolution, creating levels with items and enemies inside the rooms. They focused on a framework for game designers, where the user can customize solutions through iterations within the framework. In our

work, mission and level are also generated, but we handle less customized features. This allows us to work with less complex constraints when defining the dungeon structure and mission generation. The authors in [Alvarez et al. \(2019\)](#) did not validate their approach through a user study, while we validate the quality of the level design as well as gameplay through feedback from players.

Two EAs in [Font, Izquierdo, Manrique, and Togelius \(2016\)](#) evolve high and low-level representations for a dungeon, based on context-free grammars. The high-level EA (HL-EA) generates a numbered acyclic graph with interconnected zones, as well as the number and type of objects distributed into the rooms. The low-level EA (LL-EA) is executed for each zone, generating a cyclic graph that contains the rooms, their connections, and objects placed in each of them. The algorithm converged in less than 10s for all 50 runs, with the convergence reached after an acceptable number of generations, but no test with users was conducted or even an evaluation of the level diversity. The work in [Liapis \(2017\)](#) generates space and missions using also an EA approach with high and low-level representations. The high-level description, named sketch, is evolved using a human-provided fitness function similar to our approach. The tile of the generated sketch guides the evolution of each segment and, after all pieces have been evolved, they are combined to create a final dungeon. The level contains treasures and enemies and it must be able to fit properly the designer's inputs. All these features demand the satisfaction of many constraints during the evolution process. Another approach using different algorithms is presented in [Green et al. \(2019\)](#). Three different algorithms are proposed to create the layout of the level, while another three are proposed to place objects inside the level. Therefore, two different algorithms are used to create the final game content: one for the layout, another for the placement of objects. The generated levels were tested with artificial agents using the game MiniDungeons 2. The authors report a relevant expressive range for the generated levels.

The works in [Font et al. \(2016\)](#), [Liapis \(2017\)](#) and [Green et al. \(2019\)](#) generate missions and levels, which is the same approach we are proposing here. However, all these PCG methods required more than a single algorithm to create their content, while we introduce only one EA to evolve simultaneously space and mission contents. None of their contents was evaluated by players. The authors in [Font et al. \(2016\)](#) and [Liapis \(2017\)](#) did not create or use a game prototype, while [Green et al. \(2019\)](#) used a previously developed game. The solution approach in [Liapis \(2017\)](#) demanded many constraints for the generated content to be feasible. Some approaches from [Green et al. \(2019\)](#) were constraint-based, while others required agents to thoroughly play the levels to evaluate them, instead of a search-based algorithm as proposed by us.

We now review other relevant works approaching landmarks for level and mission generation. The work in [Valtchanov and Brown \(2012\)](#) evolves dungeons encoded in tree-like structures using Genetic Programming (GP) and translates them into a 2D grid, discarding nodes that could not fit the grid during the translation. Large dungeons are generated containing rooms of many shapes and sizes. The challenges inside the dungeon are event rooms that may or may not have missions. Minimum Spanning Trees (MSTs) are applied to generate semi-automatic 2D and 3D dungeons in [von Rymon Lipinski, Seibt, Roth, and Abé \(2019\)](#). The user creates a graph-based abstract level that becomes the final geometry for the dungeons by using the MSTs algorithm. The MSTs algorithm provides a cycle-free graph, where the removed cycle-forming edges are added later aiming to define tours, common in dungeon-like levels. The procedure generates levels in a handful of minutes for some 2D configurations and up to 5 h for more complex 3D structures.

The authors in [Antoniuk and Rokita \(2019\)](#) use L-system to create rooms and a hand-tailored procedure for corridors within 3D dungeons. In the L-system approach, the initial key set has eight keys that define how to change the room shape. Five generation rules are applied to expand this initial set of keys, where four of them can enforce a type of symmetry for the processed tile. The procedure to create corridors takes

into account properties such as right angle for junctions, start points leading to exits, different widths, and other specific properties of corridors to improve the gameplay. Fashion based Cellular Automata (CA) rules are evolved in pairs to create diverse and feasible terrain maps in [Ashlock and Kreitzer \(2020\)](#). The quality of the map is measured by the length of the shortest path between key-areas, and eight different fitness functions have modifiers that could reward density and diversity.

In comparison to our paper, the researches in [Valtchanov and Brown \(2012\)](#), [von Rymon Lipinski et al. \(2019\)](#), [Antoniuk and Rokita \(2019\)](#) and [Ashlock and Kreitzer \(2020\)](#) do not generate missions simultaneously with levels, except by the simple mission content proposed in [Valtchanov and Brown \(2012\)](#). These previous works do not present a game prototype to evaluate the final content generated, as well as no tests with human players were conducted to assess the quality and gameplay of such content. On the other hand, our study fills these three gaps. We generate space and mission simultaneously, introduce a game prototype, and evaluate the generated content with human players. Also, the works reported in [von Rymon Lipinski et al. \(2019\)](#) and [Antoniuk and Rokita \(2019\)](#) demand complex constraints during the content generation process. For instance, the hand-made algorithm developed in [von Rymon Lipinski et al. \(2019\)](#) for creating corridors must deal with many constraints. We avoid as much as possible the extensive use of such constraints in the encoding and decoding process of our EA. This also allows us to create space and mission contents using a single EA, while [von Rymon Lipinski et al. \(2019\)](#) and [Ashlock and Kreitzer \(2020\)](#) need more than one algorithm to ensure feasible and diverse levels. Our approach is inspired by the algorithm introduced in [Valtchanov and Brown \(2012\)](#), where the dungeon itself will hold the missions as locked doors.

There are other interesting researches over level and mission generation. [Ashlock and McGuinness \(2014\)](#) uses an EA to evolve a grid representation of a level, where each cell can be either empty, filled or left for the EA to alter. There are also special cells to denote checkpoints (special rooms). They tested two different fitness functions, both considering the distance between the entrance and the checkpoints. The algorithm was able to create a large variety of dungeons following the fitness' conditions. However, the authors did not conduct experiments with players nor created a game prototype to validate their results. The work in [Ashlock and Salge \(2019\)](#) explores different methods for creating enriched search spaces for game content, using representations such as Voronoi tilings and apoptotic cellular automata. The *do what's possible* representation is also introduced, which can lay out rooms using a tiny data structure. Dungeon maps with a large number of rooms were generated using a self-driving automaton, which drives its transitions with its output, generating a stream of bits. These bits, when read in groups to generate integers of a certain size, allow the automaton to execute decision processes about where to place rooms. The authors also report a two-dimensional cellular automaton with a majority based rule able to generate cavern-like maps. As interesting as these approaches can be, they do not generate missions with the levels and no tests with players in a game environment were reported.

Therefore, as summarized in [Table 1](#), our work aims to contribute to the PCG literature both by creating levels and missions together with an EA that handles as few constraints as possible. However, solution quality is evaluated both with metrics from literature and by players inside a game prototype.

3. Dungeon model

The present section describes how dungeons and their respective locked door missions are created. The tree structure that encodes the necessary information about dungeons and missions is described, as well as the decoding process to translate such a tree into the spatial placement.

Ensure: A new dungeon with several rooms

```

1: while toVisit.Count > 0 do
2:   actualRoom  $\leftarrow$  toVisit.Dequeue(ROOM_TYPE)
3:   actualDepth  $\leftarrow$  actualRoom.Depth
4:   if actualDepth > MAX_DEPTH then
5:     toVisit.Clear()
6:     break
7:   else
8:     prob  $\leftarrow$  Draw number to check if room will have children
9:     PROB_CHILD  $\leftarrow$  PROB_HAS_CHILD * (1 - actualDepth  $\div$  10)
10:    if prob  $\leq$  PROB_CHILD then
11:      // Draw number of actualRoom's children
12:      numberOfChildren  $\leftarrow$  draw(MAX_CHILD)
13:      directionArray  $\leftarrow$  Draw each child's direction
14:      for i = 1  $\rightarrow$  numberOfChildren do
15:        if prob < PROB_KEY then
16:          ROOM_TYPE  $\leftarrow$  KEY
17:        else if prob  $\geq$  PROB_NORMAL then
18:          ROOM_TYPE  $\leftarrow$  NORMAL
19:        else
20:          ROOM_TYPE  $\leftarrow$  LOCK
21:        end if
22:        child  $\leftarrow$  createRoom(ROOM_TYPE)
23:        //Insert new room as child of currentRoom, calculates [X,Y] position
24:        currentRoom.InsertChild(directionArray[i], child)
25:        child.Direction  $\leftarrow$  directionArray[i]
26:        toVisit.Enqueue(child)
27:      end for
28:    end if
29:  end if
30: end while

```

Fig. 1. Tree Structure-generation algorithm (Pereira et al., 2018).

3.1. Tree structure

Our dungeons and their missions are modeled after a tree representation as shown in Fig. 2a and presented in Pereira et al. (2018). Each node is a type of room. A Normal Room (NR) has nothing special inside it. A Key Room (KR) has a key inside, which is represented as a bold node. And a Locked Room (LR), which has a locked path between itself and its parent, is represented by a dashed edge. Thus, we are encoding the mission and the play-space at the same time.

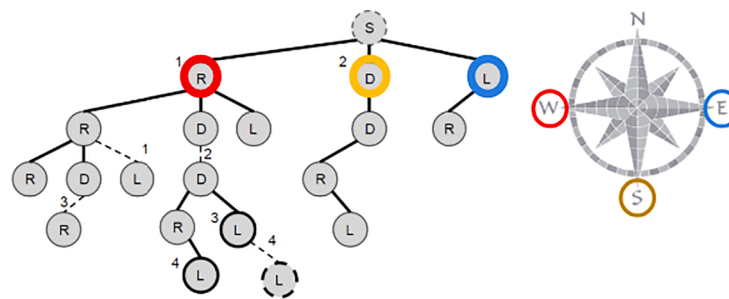
Each child node assumes a direction in a grid space related to the parent node (room) as shown in Fig. 2a: Right (R), Left (L) and Down (D). Also, the parent is always at North (N) of the room. The root node represents the player's spawn point. It is shown as the node with a thin dashed line around it in Fig. 2a. This node will always be created as the first one and placed in a queue of rooms. A breadth-first algorithm is

used to process the queue until its end and create children nodes, which are added to the queue. Each room's number of children and their respective position is determined randomly.

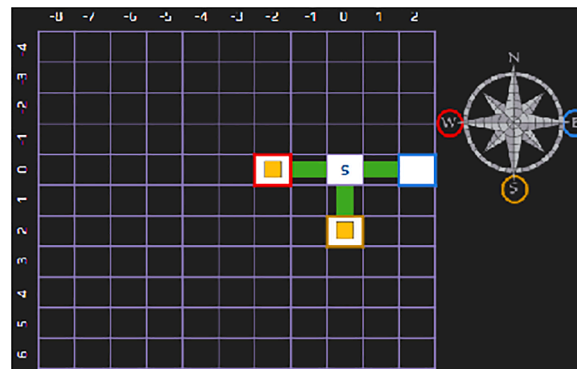
A detailed description of the tree-generation algorithm is given by Fig. 1. The breadth-first approach applied to distribute special rooms will guarantee many constraints validation by itself.

3.2. Keys and locks

The keys and locks representation follows the same as described in Pereira et al. (2018). A node may contain a key or may be locked in relation to its parent. Thus, we don't need to store information about the edges, only the nodes with their types, children, and coordinates. Each key-lock pair is unique and is given an id. Fig. 2a shows the dungeon's genotype: KR has the key id, and the LR has the id of the key that unlocks



(a) The relative directions of the nodes from the root are taken.



(b) The rotation of their direction is not necessary, as it is the same as the global orientation from the grid. Note that the keys in the rooms are represented by yellow squares.

Fig. 2. Genotype-phenotype translation. Dungeon Parameters: 20 Rooms, 4 Keys, 4 Locks and 1.5 Linear Coefficient.

it. The last LR is the destination room, represented by a thick dashed line.

3.3. The Spatial Placement: Grid

We maintain the translation from genotype to phenotype from [Pereira et al. \(2018\)](#), processing the tree into a 2D grid and placing each room and corridor according to their position from the parent node. The starting room (root node) is placed at (0,0) and the other nodes have their coordinates calculated according to their positions in the tree as shown in [Figs. 2 and 3](#). In [Fig. 2](#), the root node is not rotated, and its children can be placed in the grid in the same direction as they are in the tree. However, a rotation is done in [Fig. 3](#) to keep the parent node as north for its child. Placing a new child may lead to overlapping with existing rooms. Therefore, they are tested for overlapping when creating the tree and discarded if an overlap would happen, guaranteeing the feasibility of every tree.

The overlapping problem and its solution are better illustrated by Fig. 4. There is a tree with two overlapping nodes in Fig. 4a. When it is translated to the grid, the overlap occurs and the node that causes the overlap is removed from the grid in Figs. 4f. Thus, the tree structure is fixed and generates a feasible dungeon as shown by Fig. 4g.

4. Constrained evolutionary algorithm

Fig. 5 introduces the EA that follows the same basic steps described in Pereira et al. (2018). A new population is created by crossover and mutation operators at each generation. The tournament operator with size 2 selects individuals for reproduction, where two new individuals are created. The elitism operator will guarantee that the best individual from the previous generation is kept in the next generation.

We will refer to the EA described in the next sections as Constrained Evolutionary Algorithm (CEA) since it adds some constraints about

which tree branches can be exchanged or not during crossover and mutation. The current evolutionary method also improves the fitness function evaluation by introducing path-finding algorithms. The novelties of genetic operators and fitness function are detailed in sub-Sections 4.1 and 4.2, respectively.

4.1. Constrained crossover and mutation

Fig. 6 describes the crossover operator, where the main idea is to better preserve the semantic information of dungeons after the generation of the children solutions. The crossover algorithm keeps the same number of special rooms of the removed branch when exchanging it between parents. Moreover, the breadth-first sequence of the special room placement is preserved, since its order is saved before the exchange.

As overlapping may occur when exchanging branches, the crossover is first done in copies of the individuals. After the exchange is done, we rebuild the grid (Fig. 6, lines 12–13) and, in doing so, we may find overlapping nodes as illustrated in Fig. 4. If the remaining number of rooms, after removing overlapping nodes, is less than the necessary number of special rooms (Fig. 6, lines 14–15), the operation is canceled and another pair of cut points is selected, without replacement of the cut points. Otherwise, the crossover is validated and the copies take the place of the original levels. If no feasible cut points are found, the crossover operation does not happen, as shown in the conditional operation in line 21 in Fig. 6.

The fix algorithm redistributes the special rooms after the trade (Fig. 6, lines 24–25) by, basically, adapting some steps from Fig. 1. Instead of generating the whole tree, the fix algorithm is executed only in the individual's newly-acquired branch. At the start of the fix procedure, the probabilities of a room being an NR, KR, or LR are the same from the tree-generation algorithm, and the same visit order (breadth-first) is used. If all the special rooms saved from the previous branch

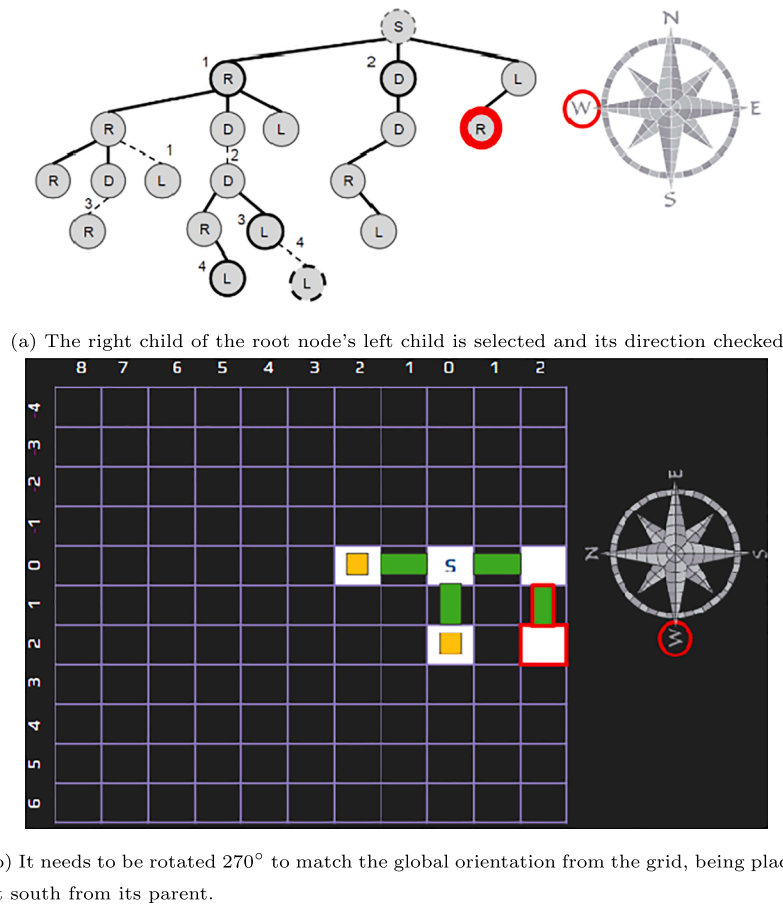


Fig. 3. Next step of genotype-phenotype translation.

have already been placed in the new one, every new room will be an NR. If there are only n rooms not yet visited by the breadth-first algorithm, during the fix execution, and n_s special rooms left, where $n = n_s$, all the n remaining rooms in the branch are changed into the n_s special rooms, sorted in the same way they were taken from their original branch.

The mutation operator is executed after crossover, if the *mutationRate* is satisfied in line 11 of Fig. 5. Next, there is a 50% chance to execute one of the two mutation operators tailor-made for the tree structure: play-space and mission mutations. If the play-space mutation is selected, there is a 50% chance to execute a leaf node addition or a leaf node exclusion. In the node-addition mutation, an NR node is inserted as a child for the selected leaf node. This change will only happen if the new node does not overlap any existing ones in the grid. In the node-exclusion mutation, the selected node will only be removed if it is an NR. Otherwise, the mutation does not occur and the node is kept in the tree.

There are also two possible operations when the mission mutation is selected: addition or change-label of a key-lock pair. To add a pair, the mutation procedure visits the tree in a breadth-first fashion, giving each visited NR node a chance to be converted into a KR. This probability is the same as the tree-creation algorithm. After the KR has been created, the next special room is an LR. If the pair is created, the algorithm stops immediately. Otherwise, it proceeds until the tree is fully visited. If no LR is created, then just a KR is added. In the change-label procedure, a KR node is randomly selected to change. Both KR and LR (if available) from the selected pair are turned into a NR.

4.2. Path-finding

All rooms and locks should be explored by the player. If they are not

explored, some of the paths of the level are not part of the locked door mission which is generally undesirable in game design. To simulate the behavior of a player traversing the dungeon created by the CEA, an adapted version of the A* and of the DFS algorithms were implemented. The A* was chosen as it is one of the most computationally efficient path-finding algorithms and it is widely used in the game industry. Its behavior simulates a player trying to reach the end of a dungeon as fast as it can, having knowledge only where the end point is. It is similar to *The Legend of Zelda* games, where the user has an item called the *Compass* which shows only the location of the dungeon's boss. Thus, the number of locks opened by the A* during its search is similar to what a skilled player who knew this information would open. Meanwhile, the DFS algorithm explores a path until a dead-end is reached. This is very similar behavior to most players who have no previous information about the dungeon and they are trying to map it. Therefore, the number of rooms visited by this algorithm would be close to the one regular players would visit.

Both algorithms must consider the rooms behind locked doors as unreachable until the KR containing its key is visited. Then, when a KR is visited by one of the path-finding approaches, the corresponding LR is immediately altered as "open" for the algorithm to visit. Its parent room, if already changed to "closed" (that is, already visited and its neighborhood explored), has its status changed to "open", allowing the algorithm to search its neighborhood again and visit the once-locked room. As the DFS algorithm does not have a fixed order to select which child to visit when a node has more than one, we select a random child that was not already visited. This adds to the behavior of different players, as each one may select a different route when playing. However, we must account for this randomness in the fitness. So we run the DFS three times, each selecting different random children to visit. Thus, A*

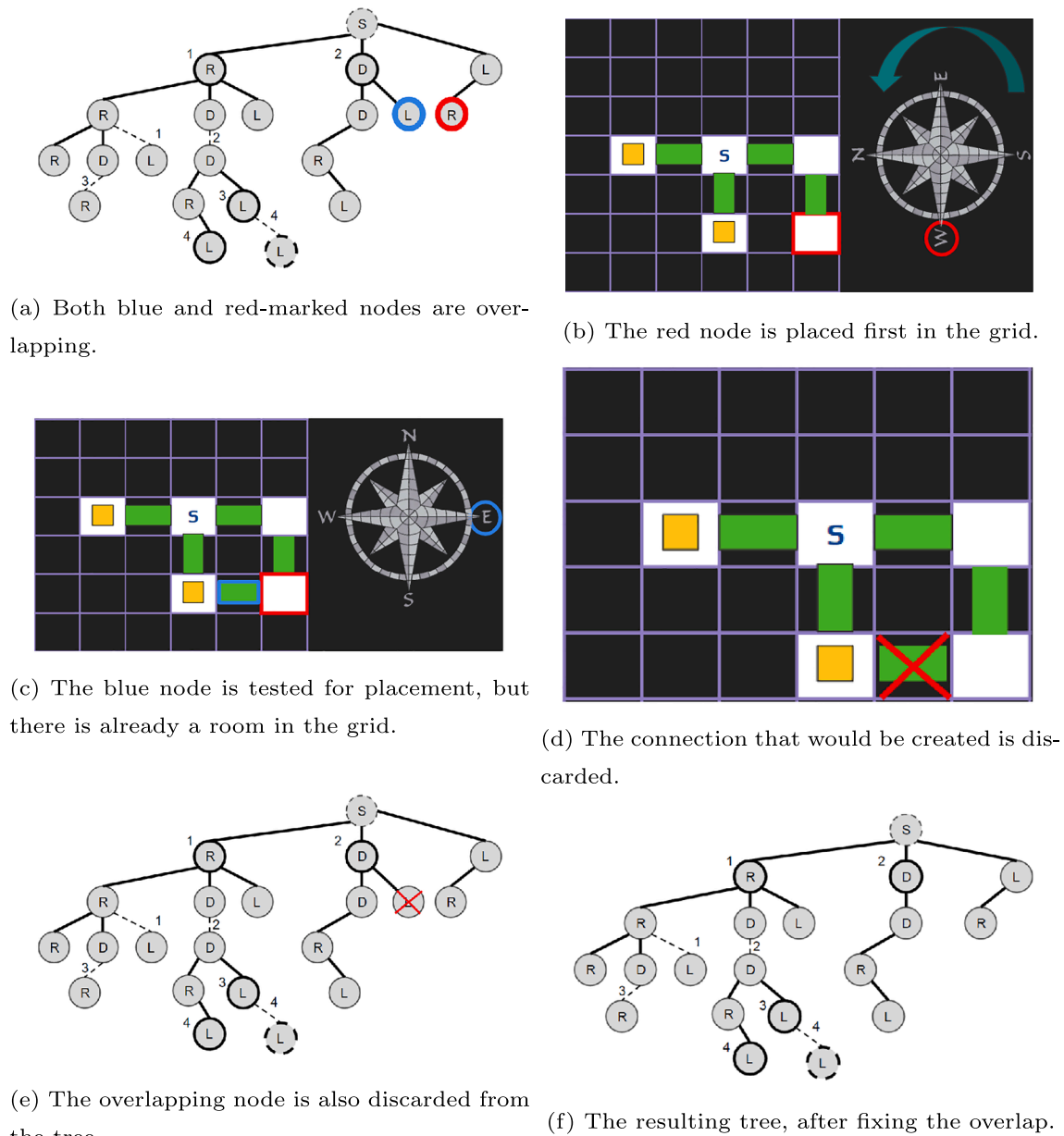


Fig. 4. Example of overlapping nodes and a fixed tree.

algorithm returns how many locks it opened while searching for the final room and the DFS returns how many rooms were visited during the search.

The fitness function evaluates if the level layout follows the desired inputs provided by the user (game designer), as well as how many of the locks the player must open to reach the end of the level and how many rooms they must visit as shown by Eq. (1). The main objective of the CEA algorithm is to create levels as close as desired by users, therefore, a double weight is given to user-related parameters.

$$f = 2(|u_r - d_r| + |u_k - d_k| + |u_l - d_l| + |u_{lin} - d_{lin}|) + \Delta_1 + \Delta_r \quad (1)$$

The user provides as input for CEA the desired number of rooms (u_r), keys (u_k), locked doors (u_l) and the level linearity (u_{lin}). CEA will search for the best level whose number of rooms (d_r), keys (d_k), locked doors (d_l) and linearity (d_{lin}) are close enough to the user's input. Therefore, the first four terms in Eq. (1) will measure how far is the generated level from the user's specification. The last two terms evaluates the exploration of the level based on outputs returned by path-finding algorithms.

Eq. (2) calculates how many rooms were explored by the DFS algorithm.

$$\Delta_r = d_r - d_r^{DFS} \quad (2)$$

Eq. (3) calculates how many locks were opened by the A* algorithm.

$$\Delta_l = d_l - d_l^{A*} \quad (3)$$

As the DFS selects random children to visit when the parent has more than one, we execute it three times for each dungeon aiming to reduce the impact of the randomness. Thus, d_r^{DFS} returns the average of the number of visited rooms by DFS, and Δ_r gives us the number of non-visited rooms. In Eq. 3, we have the difference between the total number of locks (d_l) in the level and the number of locks opened by the A* (d_l^{A*}). The linearity feature is based on the average of nodes' degree in a graph as given by Eq. 4. The parameter $nodes^{inner}$ is the number of inner nodes that the tree contains. A d_{lin} closer to 1 leads to a linear dungeon, while large d_{lin} values provide a maze-like level with more paths through multiple doors.

Require: A population *pop*, which is the list of dungeons.

Ensure: Final population after *nGenerations*.

```

1: for  $i = 0 \rightarrow nGenerations$  do
2:   newPop // Create a new empty population
3:   // Fill the new population with new individuals
4:   for  $j = 0 \rightarrow Length(pop) \div 2$  do
5:      $\lambda \leftarrow \text{draw number within } [0, 1]$ 
6:     if  $\lambda \leq crossoverRate$  then
7:        $parent1, parent2 \leftarrow Tournament(pop)$ 
8:        $child1, child2 \leftarrow Crossover(parent1, parent2)$ 
9:     end if
10:     $\lambda \leftarrow \text{draw number within } [0, 1]$ 
11:    if  $\lambda \leq mutationRate$  then
12:       $Mutate(child1, child2)$ 
13:    end if
14:     $FitnessEvaluation(child1, child2)$ 
15:    newPop.Add(child1, child2)
16:  end for
17:  bestIndividual  $\leftarrow Elitism(pop)$ 
18:  newPop.first  $\leftarrow bestIndividual$ 
19:  pop  $\leftarrow newPop$ 
20: end for

```

Fig. 5. EA - As seen in Pereira et al. (2018).

$$d_{lin} = (d_r - 1) / nodes^{inner} \quad (4)$$

5. Game prototype

We built an adventure game prototype in the Unity game engine. The game, named ProCoG (an acronym for Procedural Content Generation) is able to read a text file containing information of the dungeon generated by the CEA, and another file with the content of each room. This allows the developer to define room layouts in any way they would like to, independent of the dungeon's map. However, we did not use this second file containing the rooms' data in our experiment, as we focused solely on evaluating the quality of the dungeon structure and the missions. Keys are placed as closest to the center of KR as possible, always in an accessible place. LR has the doors between it and its parent room locked. For this game, the key-lock pairs are considered generic, meaning that any key can open any lock. But the code allows the user to create a different object for each ID.

After everything has been instantiated, the player's avatar is loaded in the starting room and the game begins. The player can move the avatar using the arrow keys or the WASD keys, and the only goal is to navigate the rooms and reach the item in the final room. A screenshot from the dungeon generated executing the game is shown in Appendix A.

The code for the game can be downloaded from Github³, and a

Windows build of the game can be downloaded at⁴.

Fig. 7 shows 4 different in-game rooms from the player's view of the game. The top-left image shows one of the starting rooms, the top-right shows the final room and the item the player must pick to finish the level. The bottom-left image contains a room with a key in its center, and the last one has a locked door. As the original Zelda games did not have unique key-lock pairs, all the dungeons in this experiment had generic keys and locks (any key can open any lock), all of them colored in yellow.

6. Computational results of the CEA

CEA was evaluated over eight sets of input parameters as shown in Tables 2–5. The first six sets of parameters explore different configurations of dungeons that are usually found in action-adventure games. The last two try to explore the upper limits of the CEA, using parameters that are not common in entertainment games. A total of 100 executions of CEA is done for each set of parameters. The crossover rate *cr* is 90%, the mutation rate *mr* = 5%, population size is 100 individuals and the number of generations is 100. The values were chosen according to previous tests and the preliminary results reported in Pereira et al. (2018). In the tree-generation (Fig. 1), the maximum depth of the tree is nine, no node may have more than three children, the probability of the room having a key is 15%, the probability of the room being locked is 15%, and the probability of it being a normal room is 70%.

The input parameters highlighted in Tables 2 and 3 are compared against those results achieved by CEA. In Table 2, it is evaluated the number of rooms (*d_r*) and keys (*d_k*) returned by the best levels of CEA against the respective highlighted input values (*u_r* and *u_k*). The number of locks (*d_l*) and linearity coefficient (*d_{lin}*) are reported in Table 3.

CEA converges in a very accurate way to the desired number of rooms, keys, and locks for the first seven input configurations. The exception is the last case, where it is returned *d_r* = 491.52 and *d_k* = 42.37 for input values *u_r* = 500 and *u_k* = 100 in Table 2. The same happens in Table 3 with *d_l* = 36.78 for *u_l* = 100. The desired linearity coefficient is close to the desired number for most cases in Table 3, but never closer than 0.04 on average. These values are worse for *u_{lin}* = 1.0, which is understandable since a linear dungeon is very unlikely to be created by our approach. However, it is interesting to observe that, even for the last set of parameters, the linearity coefficient converges to a value close to the desired one: *d_{lin}* = 1.41 when the intended one is *u_{lin}* = 1.5.

Table 4 shows that CEA generates dungeons where A* has to open every lock to reach the goal, with average values close to $\Delta_l = 0$ and standard deviations below 1.0. The last two sets of parameters are the exceptions with high average values. In these configurations, CEA does not return a dungeon where all locks are necessary to reach the goal. It can be explained by a large number of rooms, keys, and locks required, therefore, it is harder to create a unique path to the goal that enforces the use of all locks. CEA seems to have problems creating dungeons where all rooms must be visited with DFS. We have higher values on average for Δ_r than Δ_l , but it is still less than 0.5 for the first six configurations. The standard deviation is greater than 1.0 for 4 out of 8 configurations. The worst results of Δ_r happen again for the last two-parameter configurations. Moreover, these results also show that it can be a challenge for CEA to generate dungeons that must be fully explored before reaching the goal, even if the vast majority of them are required.

Table 5 reports solutions very close to the optimal value *f* = 0 for the first six parameter configuration. It makes sense based on the previous results. Thus, CEA evolves dungeons that satisfy the user's input and their rooms and locks usually must be explored to reach the goal, at least for more realistic parameter inputs. The dungeons are generated within

³ <https://github.com/Brasilia/RPG-Rooms/tree/ToG-EnglishVersion>

⁴ <https://drive.google.com/open?id=1cC0ZgiqzjmARhaVHolg5i4J6P2WJ8cN5>

Require: Two individuals $ind1$ and $ind2$ to be crossed over, cr , two lists of rooms to save the special rooms that will be exchanged between branches: $spc1$ and $spc2$.

Ensure: If crossover occurs, return two new individuals.

```

1:  $cc \leftarrow$  Draw crossover chance
2: if  $cc \leq cr$  then
3:    $CanTrade \leftarrow false$  and there are rooms left to draw in both dungeons
4:   while  $CanTrade = false$  do
5:      $roomCut1 \leftarrow$  Draw random room from dungeon  $ind1$  without replacement
6:      $roomCut2 \leftarrow$  Draw random room from dungeon  $ind2$  without replacement
7:      $saveSpecialRooms(outspc1, outspec2)$ 
8:      $clone1 \leftarrow clone(ind1)$ 
9:      $clone2 \leftarrow clone(ind2)$ 
10:     $clone1.Swap(roomCut1, roomCut2)$ 
11:     $clone2.Swap(roomCut2, roomCut1)$ 
12:     $RemakeGrid(clone1)$ 
13:     $RemakeGrid(clone2)$ 
14:    if  $RoomsInBranch(clone1, roomCut2) \geq spc1$  then
15:      if  $RoomsInBranch(clone2, roomCut1) \geq spc2$  then
16:         $CanTrade \leftarrow true$ 
17:      end if
18:    end if
19:  end while
20:  //If valid cut point exists, save the copies as new individuals
21:  if  $CanTrade = true$  then
22:     $ind1 = clone1$ 
23:     $ind2 = clone2$ 
24:     $ind1.FixBranch(roomCut2)$ 
25:     $ind2.FixBranch(roomCut1)$ 
26:  end if
27: end if
28: return  $ind1, ind2$ 

```

Fig. 6. Crossover with Constraints.

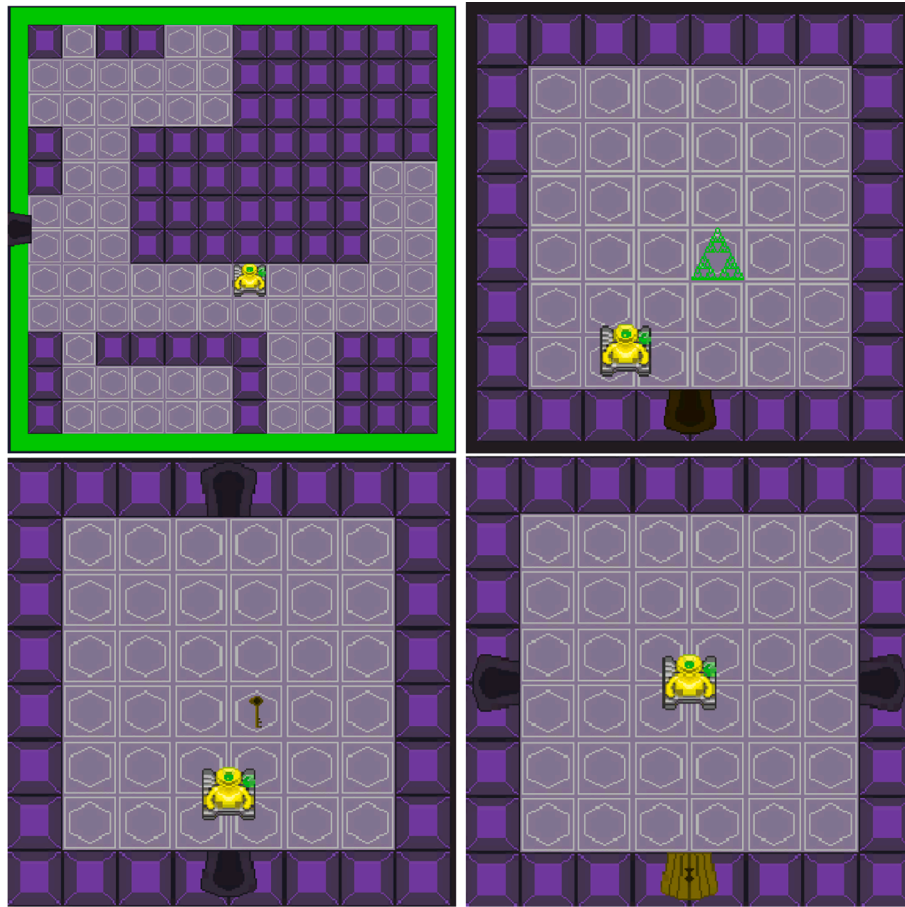


Fig. 7. Player's view in-game. Top-left: the starting room. Top-right: the final room. Bottom-left: a room with an key; Bottom-right: the room with a locked door.

Table 2

Number of rooms (d_r) and keys (d_k) obtained after 100 executions of the CEA for each combination of features.

Features (u_r, u_k, u_l, u_{lin})	d_r		d_k	
	Mean	Std. Dev.	Mean	Std. Dev.
(15, 3, 2, 2.0)	15.00	0.70	3.01	0.17
(20, 4, 4, 1.0)	20.09	0.89	4.00	0.00
(20, 4, 4, 2.0)	20.10	0.59	4.00	0.00
(25, 8, 8, 1.0)	25.00	0.14	8.00	0.00
(30, 4, 4, 2.0)	30.12	1.25	4.01	0.10
(30, 6, 6, 1.5)	29.86	1.11	6.02	0.14
(100, 20, 20, 1.5)	100.00	3.04	20.26	0.56
(500, 100, 100, 1.5)	491.52	28.87	42.37	5.26

Table 3

Number of locks (d_l) and Linear Coefficient (d_{lin}) obtained after 100 executions of the CEA for each combination of features.

Features (u_r, u_k, u_l, u_{lin})	d_l		d_{lin}	
	Mean	Std. Dev.	Mean	Std. Dev.
(15, 3, 2, 2.0)	2.01	0.17	1.96	0.12
(20, 4, 4, 1.0)	4.00	0.00	1.14	0.05
(20, 4, 4, 2.0)	4.00	0.00	1.88	0.07
(25, 8, 8, 1.0)	7.94	0.34	1.22	0.08
(30, 4, 4, 2.0)	4.00	0.00	1.88	0.10
(30, 6, 6, 1.5)	6.01	0.10	1.53	0.02
(100, 20, 20, 1.5)	19.66	0.78	1.47	0.05
(500, 100, 100, 1.5)	36.78	5.02	1.41	0.05

Table 4

Differences between number of locks in the level and opened by the A* (Δ_l) and between number of rooms in the level and visited by the DFS (Δ_r) obtained after 100 executions of the CEA for each combination of features.

Features (u_r, u_k, u_l, u_{lin})	Δ_l		Δ_r	
	Mean	Std. Dev.	Mean	Std. Dev.
(15, 3, 2, 2.0)	0.02	0.14	0.40	1.04
(20, 4, 4, 1.0)	0.01	0.10	0.13	1.17
(20, 4, 4, 2.0)	0.01	0.10	0.32	0.62
(25, 8, 8, 1.0)	0.00	0.00	0.05	0.22
(30, 4, 4, 2.0)	0.01	0.10	0.25	0.65
(30, 6, 6, 1.5)	0.11	0.78	0.41	1.79
(100, 20, 20, 1.5)	1.84	0.39	3.47	0.82
(500, 100, 100, 1.5)	5.16	4.44	16.21	6.94

Table 5

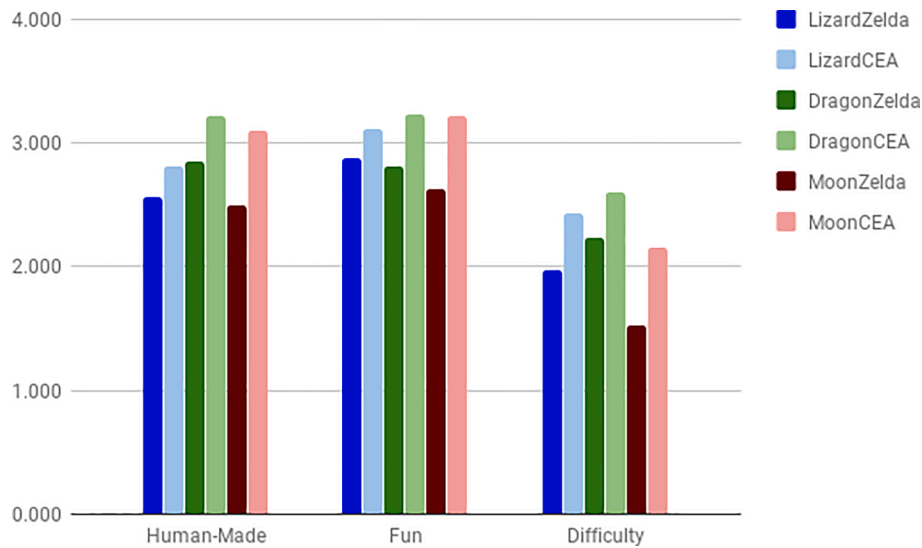
Resulting fitness (f) and execution time (Δt) in seconds obtained after 100 executions of the CEA for each combination of features.

Features (u_r, u_k, u_l, u_{lin})	f		Δt (s)	
	Mean	Std. Dev.	Mean	Std. Dev.
(15, 3, 2, 2.0)	0.08	0.22	16.45	0.75
(20, 4, 4, 1.0)	0.27	0.10	20.77	1.70
(20, 4, 4, 2.0)	0.24	0.12	18.93	0.89
(25, 8, 8, 1.0)	0.55	0.70	21.27	0.83
(30, 4, 4, 2.0)	0.25	0.28	22.28	0.69
(30, 6, 6, 1.5)	0.05	0.00	23.44	0.59
(100, 20, 20, 1.5)	1.42	1.63	72.39	2.62
(500, 100, 100, 1.5)	250.96	19.72	610.71	66.43

Table 6

Input parameters and fitness for each CEA-generated dungeon presented in ProCoG.

Dungeon	u_r	u_k	u_l	u_{lin}	f
DragonCEA	25	5	5	1.33	0.007
LizardCEA	23	6	6	1.38	0.01
MoonCEA	18	4	4	1.89	0.002

**Fig. 8.** Average Survey Answers for the Experiment with Players.

less than 25s on average for the first six parameter configurations, which is an adequate time to provide results for game designers. It may still be adequate even for online PCG, e.g., if the next dungeon is generated concurrently while the player is playing the previous one. The current algorithm employed a single thread and, if more threads were used, we would probably get a significant decrease in time, although further tests are required to quantify the real improvement.

CEA took more than 1 min for the configurations with 100 rooms, which is still a short time to generate dungeons. In the last parameter configuration, it spent more than 10 min, on average. This behavior, together with the previous bad average and standard deviation values, shows that huge dungeons can be a limitation for our algorithm. Although, it is unlikely that a game would need a similar configuration since even the one with 100 rooms is very uncommon in current video games.

7. Experiments with the game prototype

An adventure game prototype (ProCoG) was developed and described in Section 5, which allowed users to play the dungeons generated by this research. The game was temporarily uploaded to a website, and anyone with a PC's browser could access it. The link was advertised via social media without restriction for the target audience.

The user would play six different levels, three adapted directly from *The Legend of Zelda*, and three generated by our CEA using the same parameters of the level as input (number of rooms, keys, locks, and linearity coefficient). The levels adapted from the original game had only minor alterations to account for non-adjacent rooms, connected by staircases in the source (thus, teleporting the player), or rooms opened only by bombs. In the first scenario, the rooms were translated to be adjacent, and a door was placed between them to replace the staircase. No other doors were added or removed, but some rooms were rotated to avoid overlapping. For the second scenario, the destructible walls were replaced by an open door. The sequence that the levels were introduced

Table 7

F-tests followed by one-tailed t-tests for all 3 pairs of dungeons for the 3 questions in the in-game survey with the game prototype. All tests considered the answers from 70 different users and an α value of 5%.

Lizard	Human	Fun	Difficulty
F-test p-value	0.7394	0.0253	0.0220
T-test p-value	0.1213	0.0942	0.0065
Moon	Human	Fun	Difficulty
F-test p-value	0.3405	0.5013	0.3991
T-test p-value	0.0063	0.0025	0.0000
Dragon	Human	Fun	Difficulty
F-test p-value	0.9572	0.9907	0.3475
T-test p-value	0.0452	0.0151	0.0226

to the player for each part was randomized to eliminate any bias based on such sequence.

For each original *The Legend of Zelda* level, we calculated the parameters' values that were used as input by the CEA to generate analogous ones. We will mention the original levels by their names followed by the *Zelda* word (as in *MoonZelda*). Their counterpart created by our CEA will be referred to by the original level's name followed by the *CEA* word (as in *MoonCEA*). The parameter configuration for each CEA-generated level is presented in Table 6, as well as the fitness of the dungeon. All parameters of the created dungeon converged perfectly to the input value, thus, they were omitted from the table.

For each map, we saved some important gameplay data from the user after the level completion, such as the number of keys taken, the number of locks opened, the total number of rooms visited, the number of different rooms visited and the time elapsed to complete the dungeon. Also, after each dungeon was completed, the user was asked three five-

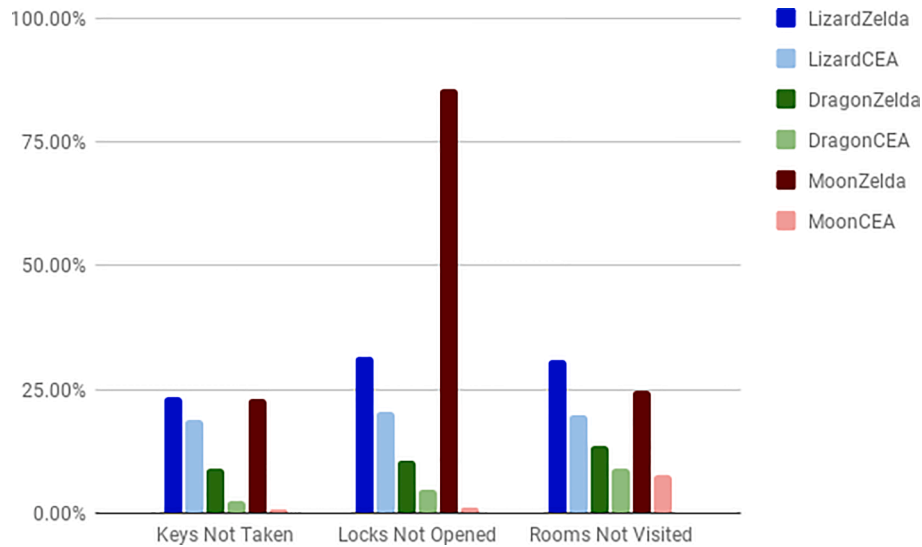


Fig. 9. Average Rooms, Locks and Keys Usage for the Experiment with Players.

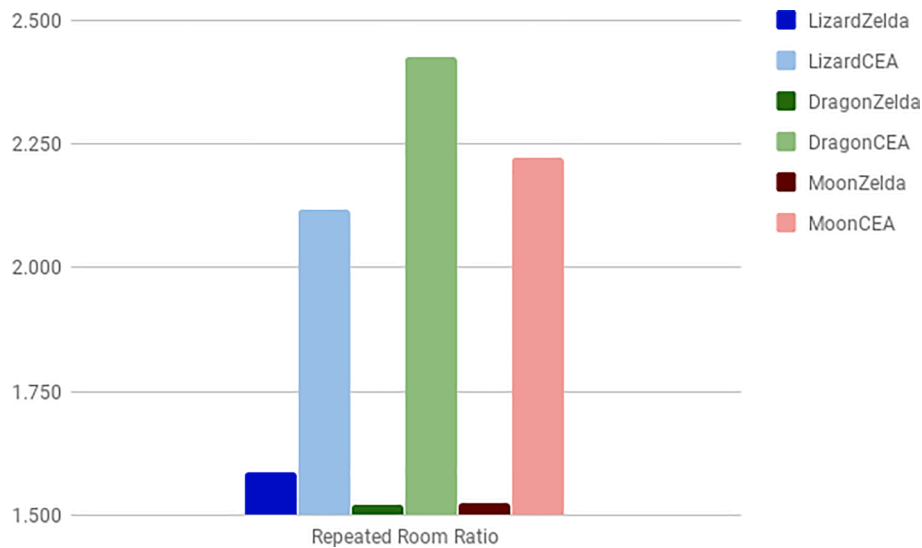


Fig. 10. Average Ratio of Room Revisiting for the Experiment with Players.

point Likert scale questions about the quality of the dungeons played. The survey was developed inside the Unity engine, making it a part of the game itself. The questions are 5-scale Likert scale statements: “This dungeon was created by a human”, “I had fun playing through this dungeon”, and “This dungeon was difficult to finish”.

All dungeons from *The Legend of Zelda* were recreated using the *Tiled Map Editor* tool⁵, and processed to adequately represent the map and rooms files which the prototype can understand. These files were then used as input for the game prototype in Unity and loaded as a dungeon.

The six different maps were sequentially played by the user, but selected in random order. To avoid bias from the configuration of rooms, all the rooms in each map were squares of 6x6 tiles. Therefore, the user goes through the same room layout all the time: identical square rooms where some of them have keys. That way, we could focus on the quality of the maps and the locked-door missions instead of room layouts. All the dungeons presented for the players are in Appendix A.

The total number of players that completed each level was different,

as some of them quit the experiment without playing all of the levels. Therefore, to evaluate all levels with the same amount of data, we used the minimum number of player-data collected for a level: 70 data points (users) for each dungeon.

First, we evaluate the results of the questionnaire. Fig. 8 shows the average answer (ranging from 1 to 5) of the users for each of the 3 questions. Each pair of the original *Zelda* dungeon and the CEA-generated counterpart share the same color hue. The former presented in a darker shade, and the latter with a lighter one. The CEA levels were all, on average, evaluated as more human-made, funnier, and difficult than the *Zelda* ones. This is a very positive result since our algorithm was able to make dungeons that looked human-made while entertaining the players. On the other hand, the rise in difficulty by itself cannot be interpreted as a positive or negative factor. But we can argue that the difficulty is one of the features that may have increased the fun in the level, considering players that enjoy challenges.

We then conduct a t-Test analysis on the data, evaluating the answers pairwise and verifying if the null hypothesis can be rejected or not. The hypothesis being that the pair of dungeon evaluated differ significantly. This test is suitable for a five-point Likert scale according to the litera-

⁵ <https://www.mapeditor.org/>

ture Winter and Dodou (2010). Using $\alpha = 5\%$, for each *Zelda*-CEA pair, we conduct a *F*-test analysis to check if the variance of both populations of answers from the same topic can be considered equal or not. If the *F*-test *p*-value is less than 0.05 in Table 7, we can conclude the variances as different. Otherwise, the null hypothesis that they are the same cannot be discarded. Therefore, we apply a homoscedastic or heteroscedastic *t*-test for the data based on the previous result.

For the *t*-test, we apply an *one-tailed t*-test with the null hypothesis being that the means of both populations are equal and the alternative hypothesis is that the mean of the population from the CEA data is greater. This can confirm if the data observed in Fig. 8, that the CEA levels had higher scores, holds true or not. If the *p*-value for the *t*-test is less than 0.05, we can conclude that the results for the CEA are statistically greater than the ones from the *Zelda* dungeons. Otherwise, they can be considered equal.

From the results in Table 7, we can conclude that the CEA was on average statistically more difficult for the *Lizard* pair of levels. However, it was considered equally developed by a human designer than the *Zelda* dungeon, as well as equally fun. On the other hand, both *Moon*'s and *Dragon*'s pair of dungeons held the same results: the CEA-generated maps were considered as more likely to have been created by a human, funnier and more difficult.

Next, we analyze the average usage of keys and locks by the players at each level, as well as the total number of different rooms visited. Fig. 9 similarly presents these data than the previous image. Each group of columns shows the average difference between the total number of available keys, locks, and unique rooms and the number of each of them taken/used/visited by the player. For every pair of dungeons, we can observe that the CEA counterparts lead to better usage of resources, leading the player to use, on average, almost every available resource on the map before completing it. At least in terms of the layout and locked-door missions, those results also confirm that the path-finding algorithms, used as part of the fitness function by the CEA, helped to create dungeons that demanded a complete exploration by the player. The results confirm what was observed in the previous results reported in Section 6, where it was presented the promising performance of the CEA to desired traits for a *Zelda*-like game.

The reported results show that the objective function was able to successfully explore the search space of our solutions, where the user-input related parameters were sufficient to allow the search for satisfactory solutions. This is supported by the players' feedback that considered the level and missions generated similar to (and sometimes better than) the ones created by humans. Also, the path-finding related parameters in the objective function led to almost complete use of resources when exploring the dungeon's structure. The path-finding in our objective function is one of the major components to sustain the users' perception that the procedural levels were made by humans, even more than their actual human-made counterparts in two cases. The objective function as a whole, and their path-finding component in particular, leads the searches for levels where keys and locks were distributed in a way that made the players explore more the dungeons, giving them the impression that the level was smartly crafted.

Another interesting aspect of our dungeons, when compared to the original *Zelda* ones, is that they demand considerably more *backtracking* through the player's path. Fig. 10 shows the average revisiting ratio: total rooms visited by the players divided by the number of unique rooms visited by them. The *Zelda* maps had an average revisiting ratio slightly above 1.5, while the CEA ones presented a ratio above 2.0, that almost reaches 2.5 for *DragonCEA*. Although backtracking may be considered a positive or negative aspect of a dungeon, depending on the many factors, it is commonly associated with the idea of a more challenging dungeon to explore. It is also associated with the positively-valued design concept of showing the players first the locked door and, next, motivating them to find the key and return to open it (Rogers, 2014, Chapter 12).

8. Conclusion

The research presented in this paper introduces the CEA as a PCG method able to generate dungeon maps and locked-door missions simultaneously. Based on the results reported, we addressed the following research question: Is it possible to generate content for mission and levels without complex constraints and loss of quality in the process? The question was addressed once we generated such content using a single EA that spent less than a minute for most cases and demands only a few numerical inputs. There is no need for creating a strict set of rules (such as grammars and their rules or sets of constraints), and the method is validated from computational tests and feedback from players through a game prototype. CEA represents the dungeon map as a tree structure, where each node was a room and each edge the connection between rooms. The rooms could belong to three different types: normal, key (which contained a key inside it), and locked (meaning the edge between it and its parent was locked). We also applied two different path-finding algorithms (*A** and DFS) to measure the exploration of the generated dungeon and to add this parameter in the fitness function. The algorithm was able to create levels that both matched the designer's inputs and placed the special rooms in a way that leads the player to explore the whole level to find the exit.

The maps generated by the CEA were saved in a file. This file and some information about the rooms were loaded in an action-adventure game prototype made with Unity and turned into a playable level. The player needed to collect the keys spread throughout the dungeon to open the colored locks and find the exit. Thus, providing a proof-of-concept that the algorithm can be applied to real games. This game prototype was populated with six different levels, three extracted from *The Legend of Zelda* game and three created by our CEA using as input parameters from each of the original dungeons. The tests with players showed that our solution was able to create enjoyable and challenging levels that were more perceived as developed by humans than by computers.

Thus, our research was able to create feasible and varied levels with locked-door missions that both matched the desired user's inputs and enforced the player to travel through (in almost all cases) the entire map to complete it, which was tested computationally for different inputs as well as by human players using our prototype. This was done with very few constraints applied to the algorithm itself, and within a very short time window (on average, less than 25s for most inputs).

One of the future steps of this work is to integrate the PCG algorithm directly into the game prototype, reducing the time to create the dungeons as they will not need to be saved in files and processed afterward by the game. Also, we plan to add more narrative elements for the game and combat mechanics for it to become more enjoyable. Next, we may release it as an entertainment game to validate the commercial value of our algorithms. We also plan to model a generic output format for the generated dungeons and a friendly interface for it in the Unity engine. Thus, we can release a *Unity package* containing our algorithms in the *Unity Asset Store*, making them available for other game developers to boost the productivity of their games.

For the map-generating EA, some of the possible extensions could be to generate special rooms filled with enemies or power-up items and test if it could also converge to the designer's input for those new parameters and place those rooms in an interesting pattern for the player. Some constraints like the ones from Horswill and Foged (2012) may be used for fitness evaluation and added to our fitness Eq. (1). The EA itself may be optimized for real-time by using threads with the correct use of semaphores to prevent data corruption. E.g.: many crossover and mutation operations can happen in different threads if they are not using the same parents, the *A** and DFS algorithms can be executed in different threads for different levels, and other optimizations.. We can add more elaborated fitness-based stop criteria such as when a specific fitness value is reached, or when there is no upgrade in the fitness after a fixed amount of generations. Another possible future work is to compare related works generating levels and missions against our proposal using

our game prototype. Also, the CEA can be evaluated under constraints related to the necessary level of exploration when executing missions, instead of 100% of exploration as employed in our experiments. This way, both Δ_r and Δ_l in our fitness function (Eq. 1) would be forced to converge to a number given by the designer corresponding to the percentage of, respectively, rooms and locks they want the players to visit/open when trying to reach the goal by the shortest path.

CRedit authorship contribution statement

Leonardo Tortoro Pereira: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing - original draft, Writing - review & editing, Visualization. **Paulo Victor de Souza Prado:** Software, Validation. **Rafael Miranda Lopes:** Software, Validation, Data curation. **Claudio Fabiano Motta Toledo:** Conceptualization, Methodology, Validation, Writing - original draft, Writing - review & editing.

Declaration of Competing Interest

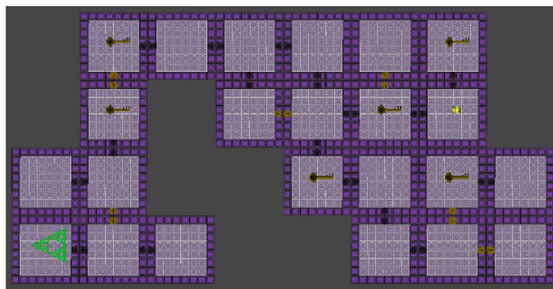
The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

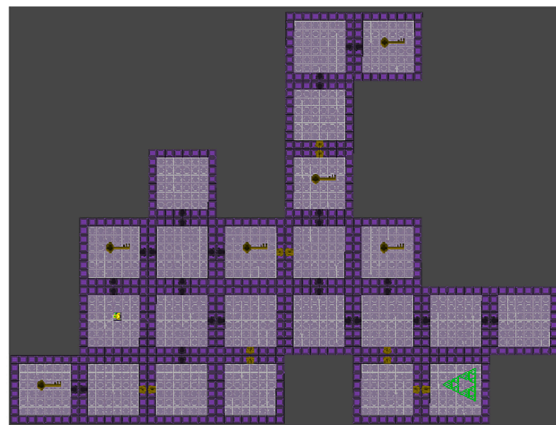
We acknowledge the National Council for Scientific and Technological Development (CNPq) for their support in funding this research.

Appendix A. Dungeons available in ProCoG

[Figs. A.11, A.12, A.13.](#)

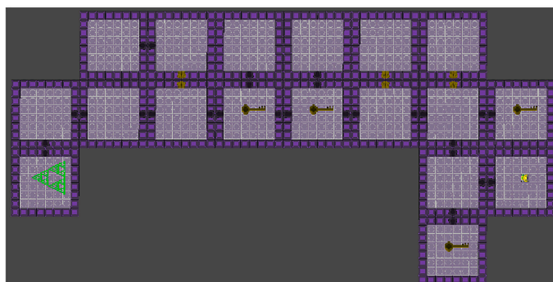


(a) Lizard Map

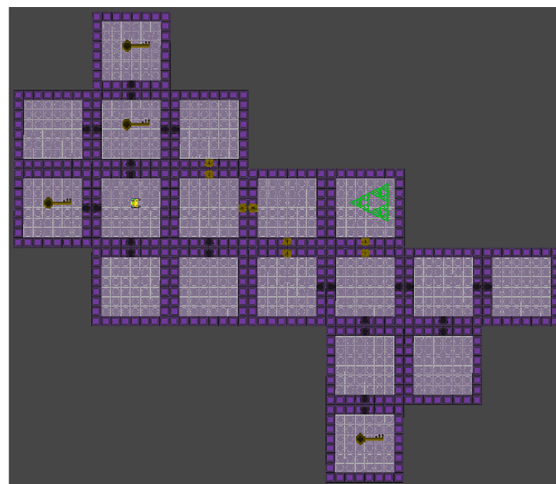


(b) Lizard Map Counterpart Created by the CEA

Fig. A.11. Lizard map adapted from The Legend of Zelda and its CEA's counterpart.

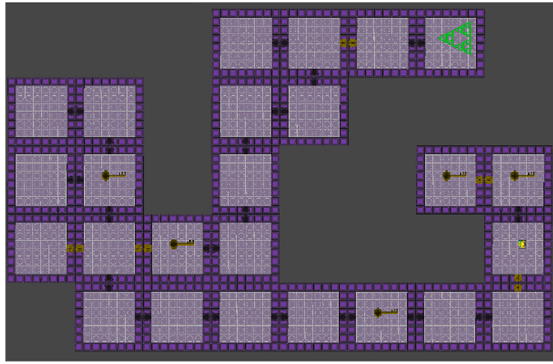


(a) Moon Map



(b) Moon Map Counterpart Created by the CEA

Fig. A.12. Moon map adapted from The Legend of Zelda and its CEA's counterpart.



(a) Dragon Map



(b) Dragon Map Counterpart Created by the CEA

Fig. A.13. Dragon map adapted from The Legend of Zelda and its CEA's counterpart.

References

- Adams, C., Parekh, H., & Louis, S. J. (2017). Procedural level design using an interactive cellular automata genetic algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion* (pp. 85–86). New York, NY, USA: Association for Computing Machinery. <https://doi.org/10.1145/3067695.3075614>.
- Adams, E. (2009). *Fundamentals of Game Design* (2nd ed.). Thousand Oaks, CA, USA: New Riders Publishing.
- Alvarez, A., Dahlskog, S., Font, J., & Togelius, J. (2019). Empowering quality diversity in dungeon design with interactive constrained map-elites. In *2019 IEEE Conference on Games (CoG)* (pp. 1–8). <https://doi.org/10.1109/CIG.2019.8848022>.
- Antoniuk, I., & Rokita, P. (2019). Procedural generation of multilevel dungeons for application in computer games using schematic maps and l-system. In R. Bembenik, L. Skonieczny, G. Protaziuk, M. Kryszkiewicz, & H. Rybinski (Eds.), *Intelligent Methods and Big Data in Industrial Applications* (pp. 261–275). Cham: Springer International Publishing. https://doi.org/10.1007/978-3-319-77604-0_19.
- Ashlock, D., & Kreitzer, M. (2020). In P. Ciancarini, M. Mazzara, A. Messina, A. Sillitti, & G. Succi (Eds.), *Evolving diverse cellular automata based level maps Proceedings of 6th International Conference in Software Engineering for Defence Applications* (pp. 10–23). Cham: Springer International Publishing.
- Ashlock, D., & McGuinness, C. (2014). Automatic generation of fantasy role-playing modules. In *2014 IEEE Conference on Computational Intelligence and Games* (pp. 1–8). <https://doi.org/10.1109/CIG.2014.6932867>.
- Ashlock, D., & Salge, C. (2019). The riddle of togelby. In *2019 IEEE Conference on Games (CoG)* (pp. 1–8).
- Baldwin, A., Dahlskog, S., Font, J. M., & Holmberg, J. (2017). Mixed-initiative procedural generation of dungeons using game design patterns. In *2017 IEEE Conference on Computational Intelligence and Games (CIG)* (pp. 25–32). <https://doi.org/10.1109/CIG.2017.8080411>.
- Calvin, A., & Michael, N. (2007). The quest in a generated world. In *DiGRA & #3907 - Proceedings of the 2007 DiGRA International Conference: Situated Play*. The University of Tokyo.
- Dahlskog, S., Togelius, J., & Björk, S. (2015). Patterns, dungeons and generators. In *FDG*.
- Dormans, J., & Bakkes, S. (2011). Generating missions and spaces for adaptable play experiences. *IEEE Transactions on Computational Intelligence and AI in Games*, 3, 216–228. <https://doi.org/10.1109/TCIAIG.2011.2149523>.
- Ferreira, L. N., & Toledo, C. F. M. (2018). Tanager: A generator of feasible and engaging levels for angry birds. *IEEE Transactions on Games*, 10, 304–316. <https://doi.org/10.1109/TCIAIG.2017.2766218>.
- Font, J. M., Izquierdo, R., Manrique, D., & Togelius, J. (2016). Constrained level generation through grammar-based evolutionary algorithms. In G. Squillero, & P. Burelli (Eds.), *Applications of Evolutionary Computation* (pp. 558–573). Cham: Springer International Publishing.
- Games, F. (2016a). Civilization vi. <https://www.civilization.com/>.
- Games, H. (2016b). No man's sky. <http://www.no-mans-sky.com/about/>.
- Green, M. C., Khalifa, A., Alsoughayer, A., Surana, D., Liapis, A., & Togelius, J. (2019). Two-step constructive approaches for dungeon generation. In *Proceedings of the 14th International Conference on the Foundations of Digital Games FDG -19*. New York, NY, USA: Association for Computing Machinery. <https://doi.org/10.1145/3337722.3341847>.
- Hendriks, M., Meijer, S., Van Der Velden, J., & Iosup, A. (2013). Procedural content generation for games: A survey. (pp. 1:1–1:22). New York, NY, USA: ACM volume 9. <https://doi.org/10.1145/2422956.2422957>.
- Horswill, I., & Foged, L. (2012). Fast procedural level population with playability constraints. In *Proceedings of the Eighth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment AIIDE'12* (pp. 20–25). AAAI Press.
- Iosup, A. (2011). Poggi: Generating puzzle instances for online games on grid infrastructures. (pp. 158–171). Chichester, UK: John Wiley and Sons Ltd. volume 23. <https://doi.org/10.1002/cpe.1638>.
- Khalifa, A., Perez-Liebana, D., Lucas, S. M., & Togelius, J. (2016). General video game level generation. In *Proceedings of the Genetic and Evolutionary Computation Conference 2016 GECCO, '16* (pp. 253–259). New York, NY, USA: ACM. <https://doi.org/10.1145/2908812.2908920>.
- Liapis, A. (2017). Multi-segment evolution of dungeon game levels. In *Proceedings of the Genetic and Evolutionary Computation Conference GECCO, '17* (pp. 203–210). New York, NY, USA: ACM. <https://doi.org/10.1145/3071178.3071180>.
- Liapis, A., Yannakakis, G. N., Nelson, M. J., Preuss, M., & Bidarra, R. (2019). Orchestrating game generation. *IEEE Transactions on Games*, 11, 48–68. <https://doi.org/10.1109/TG.2018.2870876>.
- Lora, D., Sánchez-Ruiz, A. A., González-Calero, P. A., & Gómez-Martín, M. A. (2016). Dynamic difficulty adjustment in tetris. In *The Twenty-Ninth International Flairs Conference*.
- Miyamoto, S., & Tezuka, T. (1986). The legend of zelda series. <http://www.zelda.com/>.
- Newzoo (2020). Newzoo global games market report 2020: Light version. <https://newzoo.com/insights/trend-reports/newzoo-global-games-market-report-2020-light-version/>.
- Pereira, L. T., Prado, P. V. S., & Toledo, C. (2018). Evolving dungeon maps with locked door missions. In *2018 IEEE Congress on Evolutionary Computation (CEC)* (pp. 1–8). <https://doi.org/10.1109/CEC.2018.8477718>.
- Rogers, S. (2014). *Level Up! The Guide to Great Video Game Design* ((2nd ed.)). Wiley Publishing.
- Valtchanov, V., & Brown, J. A. (2012). Evolving dungeon crawler levels with relative placement. In *Proceedings of the Fifth International C* Conference on Computer Science and Software Engineering C3S2E, '12* (pp. 27–35). New York, NY, USA: ACM. <https://doi.org/10.1145/2347583.2347587>.
- von Rymon Lipinski, B., Seibt, S., Roth, J., & Abé, D. (2019). Level graph - incremental procedural generation of indoor levels using minimum spanning trees. In *2019 IEEE Conference on Games (CoG)* (pp. 1–7). <https://doi.org/10.1109/CIG.2019.8847956>.
- Winter, J. C. F. D., & Dodou, D. (2010). *Five-point likert items: t test versus mann-whitney-wilcoxon* (p. 11). Practical Assessment, Research & Evaluation.